SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems
COURSE CODE: CSA0504

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems.* (BL3, BL6)

Activity Tool : Analytical Problem Solving (High)
Assessment Tool Weightage: 40%

QUESTIONS		
Q.No	**Question**	**Bloom's Level**
1	Given a set of relations and sample data, write SQL queries to retrieve specific information using joins and subqueries, and explain the logic used. Bloom's Level: BL3 – Apply	BL4 – Analyze
2	Using relational algebra, formulate expressions to solve complex queries such as retrieving records that satisfy multiple conditions across relations. Bloom's Level: BL4 – Analyse	BL5 – Evaluate
3	Given a real-world scenario (e.g., banking or student system), design a relational schema and construct appropriate SQL queries for data retrieval and updates. Bloom's Level: BL6 – Create	BL6 – Create
4	Analyse a given SQL query with nested subqueries and predict its output, explaining each step of execution. Bloom's Level: BL4 – Analyze	BL6 – Create
5	Compare different SQL approaches (joins vs subqueries) for solving the same problem and evaluate their efficiency. Bloom's Level: BL5 – Evaluate	BL4 – Analyze
6	Design and implement views for a database system to provide restricted data access, and justify their use for security and abstraction. Bloom's Level: BL6 – Create	BL5 – Evaluate
7	Given a database schema, write relational algebra expressions and convert them into equivalent SQL queries, analyzing differences in representation. Bloom's Level: BL4 – Analyze	BL4 – Analyze

8	Optimize a set of inefficient SQL queries by restructuring them using joins, indexing concepts, or views, and evaluate performance improvements. Bloom's Level: BL5 – Evaluate	BL5 – Evaluate
9	Develop a complex SQL query involving grouping, aggregation, and nested queries to solve a real-world problem. Bloom's Level: BL6 – Create	BL6 – Create
10	Given multiple relations, analyze the query requirements and decide whether to use views, joins, or subqueries, justifying your choice with reasoning. Bloom's Level: BL5 – Evaluate	BL5 – Evaluate

---

Code of Conduct:

I Geferlin Boffy R (192571080) *certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

ANSWERS

1) SQL Queries Using Joins and Subqueries

TABLE DEPARTMENT:

```
mysql> SELECT * FROM DEPARTMENT;
+--------+----------+
| DeptID | DeptName |
+--------+----------+
|      1 | CSE      |
|      2 | ECE      |
|      3 | EEE      |
+--------+----------+
3 rows in set (0.00 sec)
```

TABLE STUDENT:

```
mysql> SELECT * FROM STUDENT;
+-------+-------+--------+
| RegNo | Name  | DeptID |
+-------+-------+--------+
|   101 | Asha  |      1 |
|   102 | Rahul |      2 |
|   103 | Priya |      1 |
|   104 | Arun  |      3 |
|   105 | Kiran |      2 |
+-------+-------+--------+
5 rows in set (0.00 sec)
```

JOIN QUERY:

```
mysql> SELECT S.RegNo, S.Name, D.DeptName
    -> FROM STUDENT S
    -> JOIN DEPARTMENT D
    -> ON S.DeptID = D.DeptID;
+-------+-------+----------+
| RegNo | Name  | DeptName |
+-------+-------+----------+
|   101 | Asha  | CSE      |
|   103 | Priya | CSE      |
|   102 | Rahul | ECE      |
|   105 | Kiran | ECE      |
|   104 | Arun  | EEE      |
+-------+-------+----------+
5 rows in set (0.00 sec)
```

Logic

- JOIN combines rows from both tables.
- Matching is done using DeptID.

SUBQUERY:

```
mysql> SELECT Name
    -> FROM STUDENT
    -> WHERE DeptID =
    -> (
    ->     SELECT DeptID
    ->     FROM DEPARTMENT
    ->     WHERE DeptName = 'CSE'
    -> );
+-------+
| Name  |
+-------+
| Asha  |
| Priya |
+-------+
2 rows in set (0.00 sec)
```

Logic

- Inner query finds CSE department ID.
- Outer query retrieves students belonging to that depart

2) Relational Algebra Expressions

Relations:

STUDENT(RegNo, Name, DeptID)

DEPARTMENT(DeptID, DeptName)

Query 1: Retrieve all students who belong to the CSE department.

Relational Algebra:

STUDENT ⋈ STUDENT.DeptID = DEPARTMENT.DeptID DEPARTMENT

σ DeptName = 'CSE'

(STUDENT ⋈ STUDENT.DeptID = DEPARTMENT.DeptID DEPARTMENT)

Query 2: Retrieve the names of students in the CSE department.

Relational Algebra:

π Name

(

σ DeptName = 'CSE'

(STUDENT ⋈ STUDENT.DeptID = DEPARTMENT.DeptID DEPARTMENT)

)

Query 3: Retrieve RegNo and Name of students in the CSE department.

Relational Algebra:

π RegNo, Name

(

σ DeptName = 'CSE'

(STUDENT ⋈ STUDENT.DeptID = DEPARTMENT.DeptID DEPARTMENT)

)

3)Design Relational Schema

Table ACCOUNT:

```
mysql> DESC ACCOUNT;
+--------------+--------------+------+-----+---------+-------+
| Field        | Type         | Null | Key | Default | Extra |
+--------------+--------------+------+-----+---------+-------+
| AccountNo    | int          | NO   | PRI | NULL    |       |
| CustomerName | varchar(30)  | YES  |     | NULL    |       |
| Balance      | decimal(10,2)| YES  |     | NULL    |       |
+--------------+--------------+------+-----+---------+-------+
3 rows in set (0.01 sec)
```

Table TRANSACTION_DETAILS:

```
mysql> DESC TRANSACTION_DETAILS;
+-----------+--------------+------+-----+---------+-------+
| Field     | Type         | Null | Key | Default | Extra |
+-----------+--------------+------+-----+---------+-------+
| TransID   | int          | NO   | PRI | NULL    |       |
| AccountNo | int          | YES  | MUL | NULL    |       |
| Amount    | decimal(10,2)| YES  |     | NULL    |       |
| Type      | varchar(10)  | YES  |     | NULL    |       |
+-----------+--------------+------+-----+---------+-------+
4 rows in set (0.00 sec)
```

Sample account details:

```
mysql> SELECT * FROM ACCOUNT;
+-----------+--------------+----------+
| AccountNo | CustomerName | Balance  |
+-----------+--------------+----------+
|      1001 | Asha         | 25000.00 |
|      1002 | Rahul        | 18000.00 |
|      1003 | Priya        | 32000.00 |
+-----------+--------------+----------+
3 rows in set (0.00 sec)
```

Transaction Records:

```
mysql> SELECT * FROM TRANSACTION_DETAILS;
+---------+-----------+---------+--------+
| TransID | AccountNo | Amount  | Type   |
+---------+-----------+---------+--------+
|       1 |      1001 | 5000.00 | Credit |
|       2 |      1002 | 2000.00 | Debit  |
|       3 |      1003 | 3000.00 | Credit |
+---------+-----------+---------+--------+
3 rows in set (0.00 sec)
```

AccountNo 1001 now has the updated balance e.g., 30000.00:

```
mysql> SELECT * FROM ACCOUNT;
+-----------+--------------+----------+
| AccountNo | CustomerName | Balance  |
+-----------+--------------+----------+
|      1001 | Asha         | 30000.00 |
|      1002 | Rahul        | 18000.00 |
|      1003 | Priya        | 32000.00 |
+-----------+--------------+----------+
3 rows in set (0.00 sec)
```

4) Nested Subqueries

TABLES OF DEPARTMENT:

```
mysql> SELECT * FROM DEPARTMENT;
+--------+----------+
| DeptID | DeptName |
+--------+----------+
|      1 | CSE      |
|      2 | ECE      |
|      3 | EEE      |
+--------+----------+
3 rows in set (0.00 sec)
```

Output of the nested subquery:

```
mysql> SELECT Name
    -> FROM STUDENT
    -> WHERE DeptID =
    -> (
    ->     SELECT DeptID
    ->     FROM DEPARTMENT
    ->     WHERE DeptName = 'ECE'
    -> );
+-------+
| Name  |
+-------+
| Rahul |
| Kiran |
+-------+
2 rows in set (0.00 sec)
```

Inner Query:
SELECT DeptID
FROM DEPARTMENT
WHERE DeptName='ECE';
Output: DeptID = 2

Outer Query:
SELECT Name
FROM STUDENT
WHERE DeptID = 2;
Final Output:

- Rahul
- Kiran

5) Joins vs Subqueries

SOLVED USING JOIN:

```
mysql> SELECT S.Name
    -> FROM STUDENT S
    -> JOIN DEPARTMENT D
    -> ON S.DeptID = D.DeptID
    -> WHERE D.DeptName = 'CSE';
+-------+
| Name  |
+-------+
| Asha  |
| Priya |
+-------+
2 rows in set (0.00 sec)
```

SOLVED USING SUBQUERY:

```
mysql> SELECT Name
    -> FROM STUDENT
    -> WHERE DeptID =
    -> (
    ->     SELECT DeptID
    ->     FROM DEPARTMENT
    ->     WHERE DeptName = 'CSE'
    -> );
+-------+
| Name  |
+-------+
| Asha  |
| Priya |
+-------+
```

JOIN	**Subquery**
Combines tables directly	Uses one query inside another
Usually faster	May be slower
Better for multiple tables	Good for simple lookups
Preferred for large datasets	Easier for simple filtering

Conclusion

JOIN is generally more efficient than a subquery because it retrieves data by combining tables directly, making it faster for large datasets. Subqueries are simpler for filtering based on the result of another query.

6) Implementation of Database Views

Creating a VIEW:

```
mysql> SELECT * FROM STUDENT;
+-------+-------+--------+
| RegNo | Name  | DeptID |
+-------+-------+--------+
|   101 | Asha  |      1 |
|   102 | Rahul |      2 |
|   103 | Priya |      1 |
|   104 | Arun  |      3 |
|   105 | Kiran |      2 |
+-------+-------+--------+
5 rows in set (0.00 sec)
```

Display the VIEW:

```
mysql> CREATE VIEW StudentView AS
    -> SELECT RegNo, Name
    -> FROM STUDENT;
Query OK, 0 rows affected (0.01 sec)

mysql> SELECT * FROM StudentView;
+-------+-------+
| RegNo | Name  |
+-------+-------+
|   101 | Asha  |
|   102 | Rahul |
|   103 | Priya |
|   104 | Arun  |
|   105 | Kiran |
+-------+-------+
5 rows in set (0.00 sec)
```

Uses of Views:

- Provides **security** by hiding sensitive columns.
- Simplifies complex SQL queries.
- Provides **data abstraction**.
- Allows users to access only the required data.
-

7) Relational Algebra and SQL

Database Schema
STUDENT(RegNo, Name, DeptID)
DEPARTMENT(DeptID, DeptName)

Relational Algebra Expression
π Name
(
σ DeptName = 'CSE'
(STUDENT ⋈ STUDENT.DeptID = DEPARTMENT.DeptID DEPARTMENT)
)

Equivalent SQL Query

```
SELECT S.Name
FROM STUDENT S
JOIN DEPARTMENT D
ON S.DeptID = D.DeptID
WHERE D.DeptName = 'CSE';
```

Output of the SQL query

```
mysql> SELECT S.Name
    -> FROM STUDENT S
    -> JOIN DEPARTMENT D
    -> ON S.DeptID = D.DeptID
    -> WHERE D.DeptName = 'CSE';
+-------+
| Name  |
+-------+
| Asha  |
| Priya |
+-------+
2 rows in set (0.00 sec)
```

Difference between Relational Algebra and SQL

Relational Algebra	**SQL**
Theoretical query language	Practical query language
Uses symbols (σ, π, ⋈)	Uses SQL keywords (SELECT, FROM, JOIN)
Describes what operation to perform	Used to execute queries on the database

8) SQL Query Optimization

Inefficient Query (Using Subquery):

```
mysql> SELECT Name
    -> FROM STUDENT
    -> WHERE DeptID IN
    -> (
    ->     SELECT DeptID
    ->     FROM DEPARTMENT
    ->     WHERE DeptName='CSE'
    -> );
+-------+
| Name  |
+-------+
| Asha  |
| Priya |
+-------+
2 rows in set (0.00 sec)
```

Optimized Query (Using JOIN):

```
mysql> SELECT S.Name
    -> FROM STUDENT S
    -> JOIN DEPARTMENT D
    -> ON S.DeptID = D.DeptID
    -> WHERE D.DeptName='CSE';
+-------+
| Name  |
+-------+
| Asha  |
| Priya |
+-------+
2 rows in set (0.00 sec)
```

Indexing

CREATE INDEX idx_deptid
ON STUDENT(DeptID);

Explanation:

- An index speeds up searching based on DeptID.
- It improves query performance by reducing the time needed to locate matching rows.

PERFOMNCE EVALUATION

Inefficient Query	**Optimized Query**
Uses a subquery	Uses JOIN
May execute more slowly	Generally faster
Harder for large datasets	Better for large datasets
Less efficient	More efficient

Conclusion

The JOIN-based query is generally more efficient than the subquery because it combines related tables directly. Creating an index on DeptID further improves performance by allowing MySQL to find matching records more quickly.

9) Grouping, Aggregation and Nested Queries

GRUPING QUERY:

```
mysql> SELECT DeptID, COUNT(*) AS TotalStudents
    -> FROM STUDENT
    -> GROUP BY DeptID;
+--------+---------------+
| DeptID | TotalStudents |
+--------+---------------+
|      1 |             2 |
|      2 |             2 |
|      3 |             1 |
+--------+---------------+
3 rows in set (0.00 sec)
```

Aggregation with HAVING:

```
mysql> SELECT DeptID, COUNT(*) AS TotalStudents
    -> FROM STUDENT
    -> GROUP BY DeptID
    -> HAVING COUNT(*) > 1;
+--------+---------------+
| DeptID | TotalStudents |
+--------+---------------+
|      1 |             2 |
|      2 |             2 |
+--------+---------------+
2 rows in set (0.00 sec)
```

NESTED QUERY:

```
mysql> SELECT Name
    -> FROM STUDENT
    -> WHERE DeptID =
    -> (
    ->     SELECT DeptID
    ->     FROM DEPARTMENT
    ->     WHERE DeptName = 'CSE'
    -> );
+-------+
| Name  |
+-------+
| Asha  |
| Priya |
+-------+
2 rows in set (0.00 sec)
```

- **GROUP BY** groups rows based on DeptID.
- **COUNT()** counts the number of students in each department.
- **HAVING** filters grouped results.
- **Nested Query** retrieves students from the CSE department using the result of another query.

10) Views, Joins and Subqueries

JOIN Query:

```
mysql> SELECT S.RegNo, S.Name, D.DeptName
    -> FROM STUDENT S
    -> JOIN DEPARTMENT D
    -> ON S.DeptID = D.DeptID;
+-------+-------+----------+
| RegNo | Name  | DeptName |
+-------+-------+----------+
|   101 | Asha  | CSE      |
|   103 | Priya | CSE      |
|   102 | Rahul | ECE      |
|   105 | Kiran | ECE      |
|   104 | Arun  | EEE      |
+-------+-------+----------+
5 rows in set (0.00 sec)
```

SubQuery:

```
mysql> SELECT Name
    -> FROM STUDENT
    -> WHERE DeptID =
    -> (
    ->     SELECT DeptID
    ->     FROM DEPARTMENT
    ->     WHERE DeptName='ECE'
    -> );
+-------+
| Name  |
+-------+
| Rahul |
| Kiran |
+-------+
2 rows in set (0.00 sec)
```

View:

```
mysql> SELECT * FROM StudentView;
+-------+-------+
| RegNo | Name  |
+-------+-------+
|   101 | Asha  |
|   102 | Rahul |
|   103 | Priya |
|   104 | Arun  |
|   105 | Kiran |
+-------+-------+
5 rows in set (0.00 sec)
```

JUSTIFICATION

Requirement	Best Choice	Reason
Retrieve related data from multiple tables	JOIN	Efficiently combines related tables.
Find records based on another query	Subquery	Useful when one query depends on another.
Restrict user access to selected columns	VIEW	Provides security and data abstraction.

Conclusion:
JOIN is the best choice when data must be retrieved from multiple related tables. **Subqueries** are useful when one query depends on the result of another query. **Views** are best for restricting access to selected columns and simplifying frequently used queries.